ПП ПМ.05 · Методические рекомендации

Выбрана ветка FLASK

1

Один проект

Использовать свой domain.json и schema.sql из vNN. Эти файлы заменяют демонстрационные до первого запуска. Создать серверные файлы выбранной папки по законченным листингам ниже, сверяя назначение функций. HTML и CSS разрешено взять готовыми. Сопоставить имена файлов и пути. Полные листинги являются учебным образцом; собственное предметное правило и проверки обязательны. Вторую ветку не выполнять.

ПП ПМ.05 · Методические рекомендации

flask/db.py

1

Назначение и место

Соединение и атомарная транзакция. Файл расположен в service-vNN/flask/db.py. Сохранить в UTF-8. Это одна законченная единица; код перенести полностью, затем проверить синтаксис и запустить. Относительный путь к domain.json идёт из выбранной папки на один уровень вверх. Свои предметные поля не переименовывать в поля образца.

2

Полный файл

```
import os
import pymysql
from contextlib import contextmanager


@contextmanager
def database():
    connection = pymysql.connect(
        host=os.environ.get('DB_HOST', '127.0.0.1'),
        port=int(os.environ.get('DB_PORT', '3306')),
        user=os.environ.get('DB_USER', 'root'),
        password=os.environ.get('DB_PASSWORD', ''),
        database=os.environ.get('DB_NAME', 'pm05_flask'),
        charset='utf8mb4', cursorclass=pymysql.cursors.DictCursor,
    )
    try:
        yield connection
        connection.commit()
    except Exception:
        connection.rollback()
        raise
    finally:
        connection.close()


def rows(sql, args=()):
    with database() as connection:
        with connection.cursor() as cursor:
            cursor.execute(sql, args)
            return cursor.fetchall()
```

ПП ПМ.05 · Методические рекомендации

flask/rules.py

1

Назначение и место

Серверные ограничения полей и DomainPolicy. Файл расположен в service-vNN/flask/rules.py. Сохранить в UTF-8. Это одна законченная единица; код перенести полностью, затем проверить синтаксис и запустить. Относительный путь к domain.json идёт из выбранной папки на один уровень вверх. Свои предметные поля не переименовывать в поля образца.

2

Полный файл

```
import math
import re


def validate_details(domain, values):
    errors = []
    for field in domain['fields']:
        value = values.get(field['name'], '').strip()
        if not value:
            if field.get('required'):
                errors.append(f"Заполните поле «{field['label']}».")
            continue
        if len(value) > 120:
            errors.append(f"Поле «{field['label']}» слишком длинное.")
        if field['type'] == 'select' and value not in field['choices']:
            errors.append(f"Выберите допустимое значение «{field['label']}».")
        if field.get('pattern') and not re.fullmatch(field['pattern'], value):
            errors.append(f"Проверьте формат «{field['label']}».")
        if field['type'] in ('number', 'integer'):
            try:
                number = float(value)
                valid = math.isfinite(number)
                valid &= field['type'] != 'integer' or number.is_integer()
                valid &= number >= field.get('min', 0)
                valid &= number <= field.get('max', 100000)
                if not valid:
                    raise ValueError()
            except ValueError:
                errors.append(f"Проверьте число «{field['label']}».")
    for rule in domain.get('rules', []):
        condition = rule['when']
        if values.get(condition['field']) == condition['equals']:
            if rule.get('require') and not values.get(rule['require'], '').strip():
                errors.append(rule['message'])
            if rule.get('expect'):
                expected = rule['expect']
                if values.get(expected['field']) != expected['equals']:
                    errors.append(rule['message'])
    return errors


def validate_registration(values):
    errors = []
    if not re.fullmatch(r'[A-Za-z0-9]{6,40}', values.get('login', '')):
        errors.append('Логин: от 6 до 40 латинских букв и цифр.')
    if not 8 <= len(values.get('password', '')) or len(values.get('password', '
').encode('utf-8')) > 64:
        errors.append('Пароль: от 8 символов, не более 64 байт UTF-8.')
    if not re.fullmatch(r'[А-Яа-яЁё ]{3,120}', values.get('full_name', '').strip()):
        errors.append('ФИО: кириллица и пробелы.')
    if not re.fullmatch(r'8\(\d{3}\)\d{3}-\d{2}-\d{2}', values.get('phone', '')):
        errors.append('Телефон: 8(999)123-45-67.')
    if not re.fullmatch(r'[^\s@]+@[^\s@]+\.[^\s@]+', values.get('email', '')) or
len(values.get('email','')) > 120:
        errors.append('Введите корректную электронную почту.')
    return errors
```

2

Полный файл — продолжение

```
class DomainPolicy:
    def __init__(self, domain):
        self.domain = domain

    def validate(self, values):
        return validate_details(self.domain, values)
```

ПП ПМ.05 · Методические рекомендации

flask/app.py

1

Назначение и место

Сессия, роли, обработчики и отзыв. Файл расположен в service-vNN/flask/app.py. Сохранить в UTF-8. Это одна законченная единица; код перенести полностью, затем проверить синтаксис и запустить. Относительный путь к domain.json идёт из выбранной папки на один уровень вверх. Свои предметные поля не переименовывать в поля образца.

2

Полный файл

```
import json
import os
import secrets
from pathlib import Path
import pymysql
from flask import Flask, abort, redirect, render_template, request, session, url_for
from werkzeug.security import check_password_hash, generate_password_hash
from db import database, rows
from rules import DomainPolicy, validate_registration

app = Flask(__name__)
app.secret_key = os.environ.get('APP_SECRET') or secrets.token_hex(32)
app.config.update(SESSION_COOKIE_HTTPONLY=True, SESSION_COOKIE_SAMESITE='Lax',
                  SESSION_COOKIE_NAME='pm05_flask')
domain = json.loads((Path(__file__).parent.parent / '
domain.json').read_text(encoding='utf-8'))
```

2

Полный файл — продолжение

```
policy = DomainPolicy(domain)


@app.before_request
def protect():
    session.setdefault('csrf', secrets.token_hex(24))
    if request.method == 'POST' and not secrets.compare_digest(
        session['csrf'], request.form.get('csrf', '')
    ):
        abort(400, 'Недействительный токен формы. Обновите страницу.')


def current_user():
    found = rows('SELECT id,login,full_name,role FROM users WHERE id=%s',
                 (session.get('user_id', 0),))
    return found[0] if found else None


def require_user(admin=False):
    user = current_user()
    if not user or (admin and user['role'] != 'admin'):
        abort(403)
    return user


def show(page, **context):
    return render_template('page.html', page=page, domain=domain,
                           user=current_user(), values=request.form,
                           errors=context.pop('errors', []), **context)


@app.route('/register', methods=['GET', 'POST'])
def register():
    errors = []
    if request.method == 'POST':
        errors = validate_registration(request.form)
        if not errors:
            try:
                with database() as connection, connection.cursor() as cursor:
                    cursor.execute(
                        'INSERT INTO users(login,password_hash,full_name,phone,email)
 '
                        'VALUES(%s,%s,%s,%s,%s)',
                        (request.form['login'],
generate_password_hash(request.form['password']),
                         request.form['full_name'], request.form['phone'],
request.form['email']))
                return redirect(url_for('login'))
            except pymysql.err.IntegrityError:
                errors = ['Этот логин уже занят.']
    return show('register', errors=errors)


@app.route('/login', methods=['GET', 'POST'])
def login():
    errors = []
```

2

Полный файл — продолжение

```
    if request.method == 'POST':
        found = rows('SELECT * FROM users WHERE login=%s',
(request.form.get('login',''),))
        if found and check_password_hash(found[0]['password_hash'],
request.form.get('password','')):
            session.clear()
            session.update(user_id=found[0]['id'], csrf=secrets.token_hex(24))
            return redirect(url_for('index'))
        errors = ['Неверный логин или пароль.']
    return show('login', errors=errors)


@app.post('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))


@app.get('/')
def index():
    if not current_user():
        return redirect(url_for('login'))
    user = require_user()
    condition = '1=1' if user['role'] == 'admin' else 't.user_id=%s'
    args = [] if user['role'] == 'admin' else [user['id']]
    status = request.args.get('status', '')
    if status in ('Новая','В работе','Завершена'):
        condition += ' AND t.status=%s'
        args.append(status)
    tickets = rows('SELECT t.*,u.full_name,s.name AS service,n.name AS technician '
                   'FROM tickets t JOIN users u ON u.id=t.user_id '
                   'JOIN services s ON s.id=t.service_id '
                   'LEFT JOIN technicians n ON n.id=t.technician_id '
                   f'WHERE {condition} ORDER BY t.id DESC', args)
    return show('list', tickets=tickets, status=status)


@app.route('/new', methods=['GET','POST'])
def new():
    user = require_user()
    services = rows('SELECT * FROM services WHERE active=TRUE ORDER BY id')
    errors = []
    if request.method == 'POST':
        service = next((s for s in services if str(s['id']) ==
request.form.get('service_id')), None)
        if not service:
            errors.append('Выберите действующую услугу.')
        if not 3 <= len(request.form.get('equipment','').strip()) <= 120:
            errors.append('Оборудование: от 3 до 120 символов.')
        if not 5 <= len(request.form.get('symptom','').strip()) <= 2000:
            errors.append('Описание неисправности: от 5 до 2000 символов.')
        details = {f['name']:request.form.get(f['name'],'') for f in
domain['fields']}
        errors += policy.validate({**details, 'service':service['name'] if service
else ''})
        if not errors:
```

2

Полный файл — продолжение

```
            with database() as connection, connection.cursor() as cursor:
                cursor.execute('INSERT INTO
tickets(user_id,service_id,equipment,symptom,parameters,result) '
                               'VALUES(%s,%s,%s,%s,%s,%s)',
                               (user['id'],service['id'],request.form['equipment'],r
equest.form['symptom'],
                                json.dumps(details,ensure_ascii=False),''))
                ticket_id = cursor.lastrowid
                cursor.execute('INSERT INTO
ticket_history(ticket_id,actor_id,new_status,comment) '
                               'VALUES(%s,%s,%s,%s)',(ticket_id,user['id'],'Новая','
Заявка создана'))
            return redirect(url_for('detail',ticket_id=ticket_id))
    return show('new',services=services,errors=errors)


@app.route('/ticket/<int:ticket_id>', methods=['GET','POST'])
def detail(ticket_id):
    user = require_user()
    errors = []
    if request.method == 'POST':
        require_user(admin=True)
        with database() as connection, connection.cursor() as cursor:
            cursor.execute('SELECT * FROM tickets WHERE id=%s FOR
UPDATE',(ticket_id,))
            ticket = cursor.fetchone()
            if not ticket:
                abort(404)
            target = request.form.get('status','')
            expected = {'Новая':'В работе','В
работе':'Завершена'}.get(ticket['status'])
            try:
                technician = int(request.form.get('technician_id',''))
            except ValueError:
                technician = 0
            cursor.execute('SELECT id FROM technicians WHERE id=%s AND
active=TRUE',(technician or 0,))
            if not cursor.fetchone():
                errors.append('Назначьте действующего исполнителя.')
            result = request.form.get('result','').strip()
            if target != expected:
                errors.append('Недопустимый переход статуса.')
            if target == 'Завершена' and not 5 <= len(result) <= 2000:
                errors.append('Для завершения опишите результат: от 5 до 2000
символов.')
            if len(result) > 2000:
                errors.append('Результат слишком длинный.')
            if not errors:
                cursor.execute('UPDATE tickets SET
status=%s,technician_id=%s,result=%s WHERE id=%s',
                               (target,technician,result,ticket_id))
                cursor.execute('INSERT INTO
ticket_history(ticket_id,actor_id,old_status,new_status,comment) '
                               'VALUES(%s,%s,%s,%s,%s)',(ticket_id,user['id'],ticket
['status'],target,result))
    found = rows('SELECT t.*,s.name AS service,n.name AS technician FROM tickets t '
```

2

Полный файл — продолжение

```
                'JOIN services s ON s.id=t.service_id LEFT JOIN technicians n ON
n.id=t.technician_id '
                'WHERE t.id=%s AND (%s=\'admin\' OR
t.user_id=%s)',(ticket_id,user['role'],user['id']))
    if not found:
        abort(404)
    history = rows('SELECT h.*,u.full_name FROM ticket_history h JOIN users u ON
u.id=h.actor_id '
                   'WHERE ticket_id=%s ORDER BY h.id',(ticket_id,))
    return show('detail',ticket=found[0],details=json.loads(found[0]['parameters']),
history=history,
                technicians=rows('SELECT * FROM technicians WHERE
active=TRUE'),errors=errors,
                reviews=rows('SELECT body,created_at FROM ticket_reviews WHERE
ticket_id=%s',(ticket_id,)))


@app.post('/ticket/<int:ticket_id>/review')
def review(ticket_id):
    user = require_user()
    if user['role'] != 'client':
        abort(403)
    body = request.form.get('body', '').strip()
    with database() as connection, connection.cursor() as cursor:
        cursor.execute('SELECT * FROM tickets WHERE id=%s AND user_id=%s FOR
UPDATE',
                       (ticket_id, user['id']))
        ticket = cursor.fetchone()
        if not ticket:
            abort(404)
        if ticket['status'] != 'Завершена':
            abort(409, 'Отзыв доступен после завершения заявки.')
        if not 5 <= len(body) <= 1000:
            abort(400, 'Отзыв: от 5 до 1000 символов.')
        cursor.execute('SELECT id FROM ticket_reviews WHERE ticket_id=%s',
(ticket_id,))
        if cursor.fetchone():
            abort(409, 'Отзыв уже сохранён.')
        cursor.execute('INSERT INTO ticket_reviews(ticket_id,user_id,body)
VALUES(%s,%s,%s)',
                       (ticket_id,user['id'],body))
    return redirect(url_for('detail',ticket_id=ticket_id))


@app.cli.command('create-admin')
def create_admin():
    import getpass
    login = input('Логин администратора: ')
    password = getpass.getpass('Пароль администратора: ')
    with database() as connection, connection.cursor() as cursor:
        cursor.execute('INSERT INTO users(login,password_hash,full_name,phone,email,
role) '
                       'VALUES(%s,%s,%s,%s,%s,%s)',
                       (login,generate_password_hash(password),'Учебный
Администратор',
                        '8(999)123-45-67','admin@example.test','admin'))
```

2

Полный файл — продолжение

```
    print('Администратор создан в выбранной учебной базе.')
```

ПП ПМ.05 · Методические рекомендации

flask/templates/page.html

1

Назначение и место

Готовая минимальная оболочка. Файл расположен в service-vNN/flask/templates/page.html. Сохранить в UTF-8. Это одна законченная единица; код перенести полностью, затем проверить синтаксис и запустить. Относительный путь к domain.json идёт из выбранной папки на один уровень вверх. Свои предметные поля не переименовывать в поля образца.

2

Полный файл

```
<!doctype html>
<html lang="ru">
<head>
  <meta charset="utf-8"><meta name="viewport"
content="width=device-width,initial-scale=1">
  <title>{{ domain.title }} — ПМ.05</title>
  <link rel="stylesheet" href="{{ url_for('static',filename='style.css') }}">
</head>
<body><main>
<header><p>ПМ.05 · Проектирование и разработка информационных систем</p><h1>{{
domain.title }}</h1></header>
<nav>
{% if user %}
  <a href="/">{{ 'Очередь заявок' if user.role == 'admin' else 'Мои заявки' }}</a>
  <a href="/new">Создать заявку</a><span>{{ user.full_name }} · {{ user.role
}}</span>
  <form action="/logout" method="post"><input type="hidden" name="csrf" value="{{
session.csrf }}"><button>Выйти</button></form>
{% else %}<a href="/login">Вход</a><a href="/register">Регистрация</a>{% endif %}
</nav>
{% if errors %}<section class="error" role="alert"><h2>Проверьте введённые
данные</h2><ul>{% for error in errors %}<li>{{ error }}</li>{% endfor
%}</ul></section>{% endif %}
```

2

Полный файл — продолжение

```
{% macro input(name,label,type='text') %}
<label>{{ label }}<input name="{{ name }}" type="{{ type }}" value="{{ '' if type
== 'password' else values.get(name,'') }}" required></label>
{% endmacro %}
{% if page in ['login','register'] %}
<section><h2>{{ 'Регистрация клиента' if page == 'register' else 'Вход в систему'
}}</h2>
<form method="post"><input type="hidden" name="csrf" value="{{ session.csrf }}">
{{ input('login','Логин') }}{{ input('password','Пароль','password') }}
{% if page == 'register' %}{{ input('full_name','ФИО') }}{{ input('phone','Телефон:
8(999)123-45-67') }}{{ input('email','Электронная почта','email') }}{% endif %}
<button>{{ 'Создать пользователя' if page == 'register' else 'Войти'
}}</button></form>
<p><a href="{{ '/login' if page == 'register' else '/register' }}">{{ 'Уже
зарегистрированы? Вход' if page == 'register' else 'Ещё не зарегистрированы?
Регистрация' }}</a></p></section>
{% elif page == 'new' %}
<section><h2>Новая заявка</h2><form method="post"><input type="hidden" name="csrf"
value="{{ session.csrf }}">
{{ input('equipment','Оборудование и модель') }}
<label>Неисправность<textarea name="symptom" required>{{ values.get('symptom','')
}}</textarea></label>
<label>Услуга<select name="service_id" required><option value="">Выберите
услугу</option>{% for s in services %}<option value="{{ s.id }}" {% if
values.get('service_id') == s.id|string %}selected{% endif %}>{{ s.name
}}</option>{% endfor %}</select></label>
{% for f in domain.fields %}<label>{{ f.label }}
{% if f.type == 'select' %}<select name="{{ f.name }}"><option value="">Выберите
значение</option>{% for choice in f.choices %}<option {% if values.get(f.name) ==
choice %}selected{% endif %}>{{ choice }}</option>{% endfor %}</select>
{% else %}<input name="{{ f.name }}" value="{{ values.get(f.name,'') }}" type="{{ '
number' if f.type in ['number','integer'] else 'text' }}" step="{{ '1' if f.type == '
integer' else 'any' }}">{% endif %}</label>{% endfor %}
<button>Отправить заявку</button></form></section>
{% elif page == 'list' %}
<section><h2>{{ 'Очередь заявок' if user.role == 'admin' else 'Мои заявки' }}</h2>
<form method="get"><label>Статус<select name="status"><option
value="">Все</option>{% for s in ['Новая','В работе','Завершена'] %}<option {% if
status == s %}selected{% endif %}>{{ s }}</option>{% endfor
%}</select></label><button>Применить фильтр</button></form>
{% for t in tickets %}<article><h3><a href="/ticket/{{ t.id }}">Заявка №{{ t.id }}
· {{ t.equipment }}</a></h3><p>{{ t.service }} · <strong>{{ t.status
}}</strong></p><p>Клиент: {{ t.full_name }} · Исполнитель: {{ t.technician or 'Не
назначен' }}</p></article>{% else %}<p>Заявок по выбранным условиям нет.</p>{%
endfor %}</section>
{% elif page == 'detail' %}
<section><h2>Заявка №{{ ticket.id }} · {{ ticket.status }}</h2><p>{{
ticket.equipment }} · {{ ticket.service }}</p><p>{{ ticket.symptom }}</p>
<dl>{% for f in domain.fields %}<dt>{{ f.label }}</dt><dd>{{ details.get(f.name) or '
Не указано' }}</dd>{% endfor %}</dl>
<p>Исполнитель: {{ ticket.technician or 'Не назначен' }}</p><p>Результат: {{
ticket.result or 'Работа ещё не завершена' }}</p>
{% if user.role == 'admin' and ticket.status != 'Завершена' %}
<form method="post"><input type="hidden" name="csrf" value="{{ session.csrf }}">
```

2

Полный файл — продолжение

```
<label>Исполнитель<select name="technician_id"><option value="">Назначьте
исполнителя</option>{% for n in technicians %}<option value="{{ n.id }}" {% if
ticket.technician_id == n.id %}selected{% endif %}>{{ n.name }}</option>{% endfor
%}</select></label>
<input type="hidden" name="status" value="{{ 'В работе' if ticket.status == 'Новая'
else 'Завершена' }}">
<label>Описание результата<textarea name="result">{{
values.get('result',ticket.result) }}</textarea></label>
<button>{{ 'Принять в работу' if ticket.status == 'Новая' else 'Завершить заявку'
}}</button></form>{% endif %}</section>
<section><h2>История обработки</h2>{% for h in history %}<article><p>{{
h.changed_at }} · {{ h.full_name }}</p><p>{{ h.old_status or 'Создание' }} →
<strong>{{ h.new_status }}</strong></p><p>{{ h.comment }}</p></article>{% endfor
%}</section>
{% endif %}

{% if page == 'detail' %}
<section><h2>Отзыв о работе</h2>
{% for r in reviews %}<p>{{ r.body }}</p><p>{{ r.created_at }}</p>{% else %}
{% if user.role == 'client' and ticket.status == 'Завершена' %}
<form method="post" action="/ticket/{{ ticket.id }}/review">
<input type="hidden" name="csrf" value="{{ session.csrf }}">
<label>Отзыв<textarea name="body" required minlength="5"
maxlength="1000"></textarea></label>
<button>Сохранить отзыв</button></form>
{% else %}<p>Отзыв доступен владельцу после завершения заявки.</p>{% endif %}
{% endfor %}</section>{% endif %}
</main></body></html>
```

ПП ПМ.05 · Методические рекомендации

flask/static/style.css

1

Назначение и место

Стили оболочки. Файл расположен в service-vNN/flask/static/style.css. Сохранить в UTF-8. Это одна законченная единица; код перенести полностью, затем проверить синтаксис и запустить. Относительный путь к domain.json идёт из выбранной папки на один уровень вверх. Свои предметные поля не переименовывать в поля образца.

2

Полный файл

```
:root{font-family:Arial,sans-serif;color:#18252a;background:#edf2f1;line-height:1.5}
*{box-sizing:border-box}main{max-width:1000px;margin:auto;padding:28px}
header{border-bottom:5px solid #136653}h1{font-size:30px}h2{font-size:23px}
nav{display:flex;gap:20px;flex-wrap:wrap;align-items:center;padding:20px 0}
a{color:#095749}section{background:white;padding:25px;margin:18px
0;border-radius:8px}
form{max-width:620px}nav form{margin:0}label{display:block;font-weight:bold;margin:1
2px 0}
input,select,textarea{display:block;width:100%;font:inherit;padding:10px;border:1px
solid #889e97;border-radius:4px}
textarea{min-height:95px}button{background:#136653;color:white;border:0;border-radiu
s:4px;padding:11px 20px;font:inherit;cursor:pointer}
button:focus-visible,a:focus-visible{outline:3px solid #d79117;outline-offset:3px}
article{border-left:4px solid #a5c5b9;padding:4px 18px;margin:14px
0;background:#f4f8f6}
.error{background:#fff3ed;border:2px solid #a5361a}dt{font-weight:bold}dd{margin-lef
t:0;margin-bottom:10px}
footer{color:#536760;margin:24px 0}@media(max-width:600px){main{padding:12px}section
{padding:16px}h1{font-size:24px}}
```

ПП ПМ.05 · Методические рекомендации

flask/requirements.txt

1

Назначение и место

Зависимости. Файл расположен в service-vNN/flask/requirements.txt. Сохранить в UTF-8. Это одна законченная единица; код перенести полностью, затем проверить синтаксис и запустить. Относительный путь к domain.json идёт из выбранной папки на один уровень вверх. Свои предметные поля не переименовывать в поля образца.

2

Полный файл

```
Flask==3.1.3
PyMySQL==1.2.0
```

Проверка синтаксиса и запуск

1

Терминал выбранного проекта

Открыть PowerShell в папке flask своего проекта. Для Flask нужен Python 3.11+, для PHP — PHP 8.2+ с pdo_mysql, mbstring и JSON; MySQL/MariaDB и пустая собственная БД уже подготовлены. NN заменить номером, реквизиты — значениями проверенного подключения DBeaver. Переменные задавать в том же окне перед запуском; не копировать чужой пароль. Для PHP использовать установленный путь php.exe, если он отличается от XAMPP.

2

Подготовка окружения

```
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

3

Проверка файлов

```
.\.venv\Scripts\python.exe -m py_compile app.py db.py rules.py
```

4

Подготовка и запуск

```
$env:DB_HOST="127.0.0.1"
$env:DB_PORT="3306"
$env:DB_NAME="pm05_vNN"
$env:DB_USER="учебный_пользователь"
$env:DB_PASSWORD="учебный_пароль"
.\.venv\Scripts\python.exe -m flask --app app create-admin
.\.venv\Scripts\python.exe -m flask --app app run --port 5057
```

5

Результат и ошибки

В create-admin ввести adminpm05 и вымышленный пароль Practice123; выполнять один раз на новой БД. При duplicate login не повторять создание на старой БД. Открыть http://127.0.0.1:5057/register. После изменения Python остановить свой сервер Ctrl+C и запустить вновь. При ModuleNotFoundError установить requirements именно в .venv. Для PHP проверить модули командой -m; ошибка could not find driver означает отсутствие pdo_mysql. При HTTP 500 посмотреть сообщение в терминале сервера; проверить подключение, таблицы и операцию, затем исправить причину и повторить. При смене БД создать нового администратора. Терминал оставить работающим до конца сценариев. Учебный сервер слушает только 127.0.0.1.

ПП ПМ.05 · Методические рекомендации

Серверный контракт и прямой POST

1

Маршруты

GET/POST /register, /login; POST /logout; GET /; GET/POST /new; GET/POST /ticket/<id>; POST /ticket/<id>/review. Регистрация: login/password/full_name/phone/email. Новая заявка: equipment/symptom/service_id и поля domain.fields. Переход: status/technician_id/result. Отзыв: body. Каждый POST содержит csrf из текущей сессии. Успешная регистрация, вход, создание и отзыв перенаправляют 302; форма с предметной ошибкой возвращает 200 с сообщением и без записи. Чужая карточка — 404, запрещённая роль — 403, CSRF — 400. Отзыв: ранний/повторный — 409, длина — 400. История сохраняется при успешных статусных действиях.

2

Пример в Console браузера после входа

```
const csrf = document.querySelector('input[name=csrf]').value;
const data = new URLSearchParams({csrf,service_id:'99999',
  equipment:'Учебное изделие',symptom:'Проверка ошибочного запроса'});
const response = await fetch('/new',{method:'POST',body:data});
console.log(response.status, await response.text());
```

3

Ожидание

Пример отправляет неизвестную услугу и должен показать серверную ошибку формы. Числа tickets и ticket_history остаются прежними. Для своих предметных случаев добавить поля из case.json. Не выполнять консольный пример на сторонних сайтах. Передавать контракт вместе с README и своим кодом.